

Lakeflow Declarative Pipelines

From Code to Flows





The Pains of ETL



Glue code everywhere.
Quality bolted on.
Pipelines break

March 2026

Migration

- Upgrading your Azure Data Factory pipelines to Fabric. [Learn more.](#)
- Upgrading your Azure Synapse Analytics pipelines to Fabric. [Learn more.](#)

January 2026

Migration

Assessing your Azure Data Factory and Synapse pipelines for migration to Fabric. [Learn more.](#)

December 2025

Data movement

HDInsight Hive activity now supports Managed Identity authentication to ADLS Gen2 Storage accounts for Bring Your Own Compute. [Learn more.](#)

October 2025

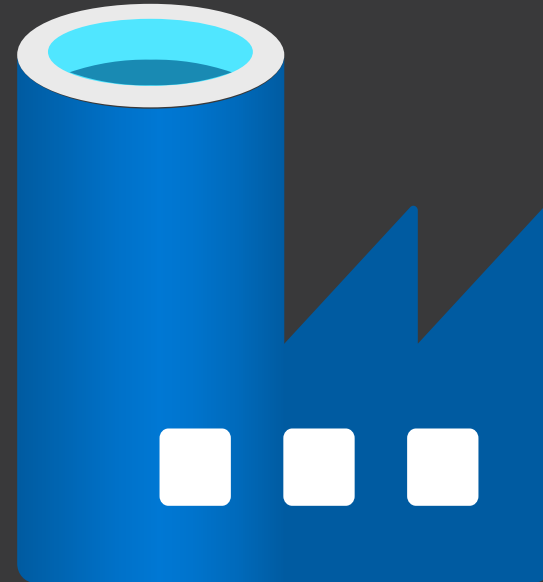
Data movement

HDInsight Hive activity now supports Managed Identity authentication to ADLS Gen2 Storage accounts for Bring Your Own Compute. [Learn more.](#)

August 2025

Data flow

We're updating Azure Data Factory Mapping Data Flows to use to use Spark 3.4.



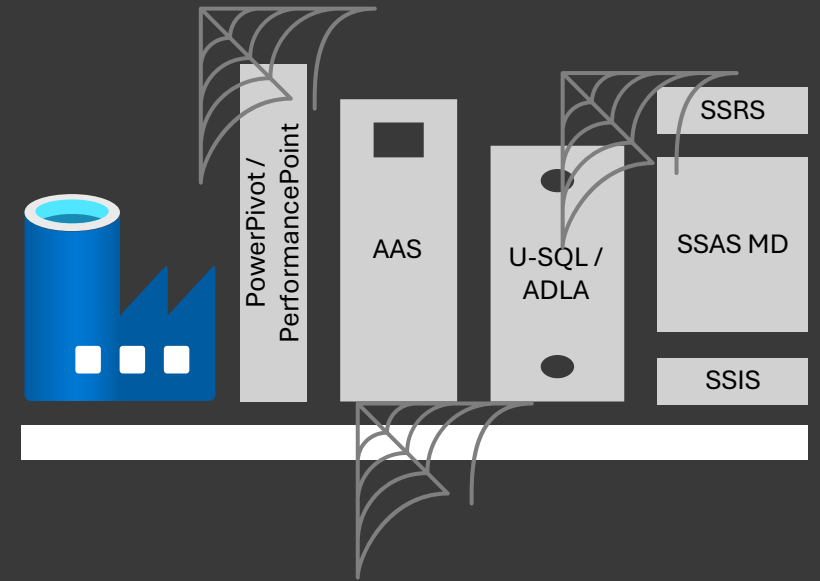
Is Azure Data Factory (ADF) Dead?

INTRODUCTION

Is Azure Data Factory (ADF) Dead?



Lakeflow





Falek Miah

Chief Architect



~20 Years in Data, Analysis & AI



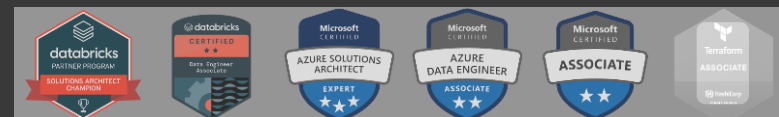
Intensive Data Engineering & Analytics Experience



Data, Cloud & DevOps Enthusiast



Microsoft Azure, Databricks (Spark), Terraform (HashiCorp) certified





THE PROBLEM



Problems Trying to Address

WHAT YOU END UP WITH



Operational Complexity

Complex pipelines
Separate orchestration jobs for every step



Manual Processes

Custom retry and error handling logic
Manual dependency ordering



Data Quality Issues

Data quality checks bolted on as an afterthought

THE PROBLEM

Problems Trying to Address

WHAT YOU END UP WITH



Operational Complexity

Complex pipelines
Separate orchestration jobs for every step



Manual Processes

Custom retry and error handling logic
Manual dependency ordering



Data Quality Issues

Data quality checks bolted on as an afterthought



DATABRICKS LAKEFLOW



Operational Complexity

Declare what the data should look like
Simplify development efforts



Manual Processes

Platform resolve dependencies
Handles and unifies Batch and Streaming datasets



Data Quality Issues

Quality expectations built into flows



Lakeflow



INTRODUCING LAKEFLOW



A unified solution for ingestion, transformation & orchestration



Lakeflow Connect

Managed connectors for SaaS apps, databases and cloud storage. Incremental by default. No custom code.



Spark Declarative Pipelines (SDP)

Define flows in SQL or Python. The platform resolves dependencies, manages retries, and handles incremental processing. Replaced DLT.



Lakeflow Jobs

Orchestrate and monitor every pipeline and workflow from one place. Powered by serverless compute. Replace Workflows.

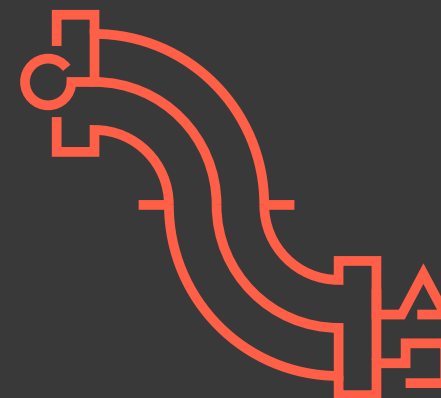


LAKEFLOW SPARK DECLARATIVE PIPELINES



What Are Lakeflow Spark Declarative Pipelines?

- ❖ Simple **declarative** framework to build Data Pipelines (ELT)
- ❖ Understands **data dependencies** in real-time
- ❖ It has built in **data quality** (Expectations)
- ❖ Allows data professionals to worry less about **operations** and more about getting **quality data**



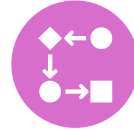


Core Component



Flow

Reads from a source, applies logic, writes to a target. Supports batch and streaming.



Dependency Management

Pipeline analyses the dependencies and builds the execution order automatically.



Streaming Table

Append-only incremental target. Ideal for ingestion and low-level transformations, such as Bronze and Silver.



Materialized View

Batch target. Pre-computed and incrementally refreshed. Ideal for Gold layer aggregations and analytics.



Expectations

Built-in data quality enforcement - warn, drop, or fail.



AUTO CDC

Unique to Databricks. Handles out-of-order events, deduplication, & supports both SCD Type 1 and Type 2 outputs.

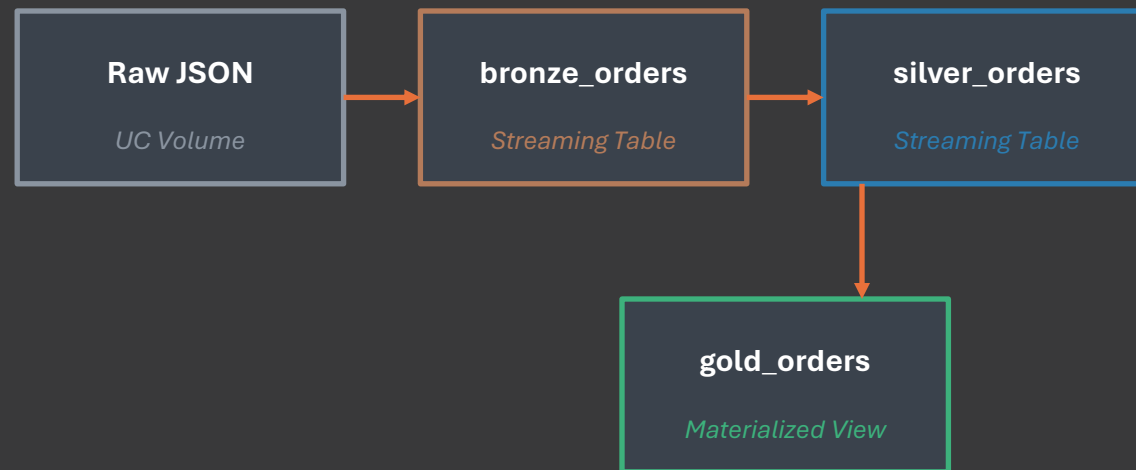


Governance Built In, Not Bolted On

Unity Catalog and lineage from day one!

- Every Lakeflow table sits in Unity Catalog by default
- Permissions, audit, and discovery - out of the box
- Lineage is auto-generated as the pipeline runs
- Column-level lineage from source to gold
- One place to govern data and AI assets

LINEAGE — AUTO GENERATED





Simple End-to-End Pipeline

```
-- Bronze: stream raw JSON
CREATE OR REFRESH STREAMING TABLE bronze_orders
AS
SELECT *
FROM STREAM read_files(
  '/uc/volumes/raw/orders',
  format => 'json',
  multiLine => true
);

-----

-- Silver: clean & type
CREATE OR REFRESH STREAMING TABLE silver_orders
AS
SELECT
  order_id::INT, customer_id,
  amount::DECIMAL(10,2)
FROM STREAM(bronze_orders)
WHERE order_id IS NOT NULL;

-----

-- Gold: aggregation
CREATE OR REFRESH MATERIALIZED VIEW gold_orders_by_date
AS
SELECT date, SUM(amount)
FROM silver_orders
GROUP BY date;
```

Demo - Simple Pipeline

- Handful of **SQL** statements
- **STREAM** keyword enables incremental processing
- **Dependencies** resolved automatically
- The platform builds the **DAG**
- Materialized View is **incrementally refreshed**



Data Quality Expectation

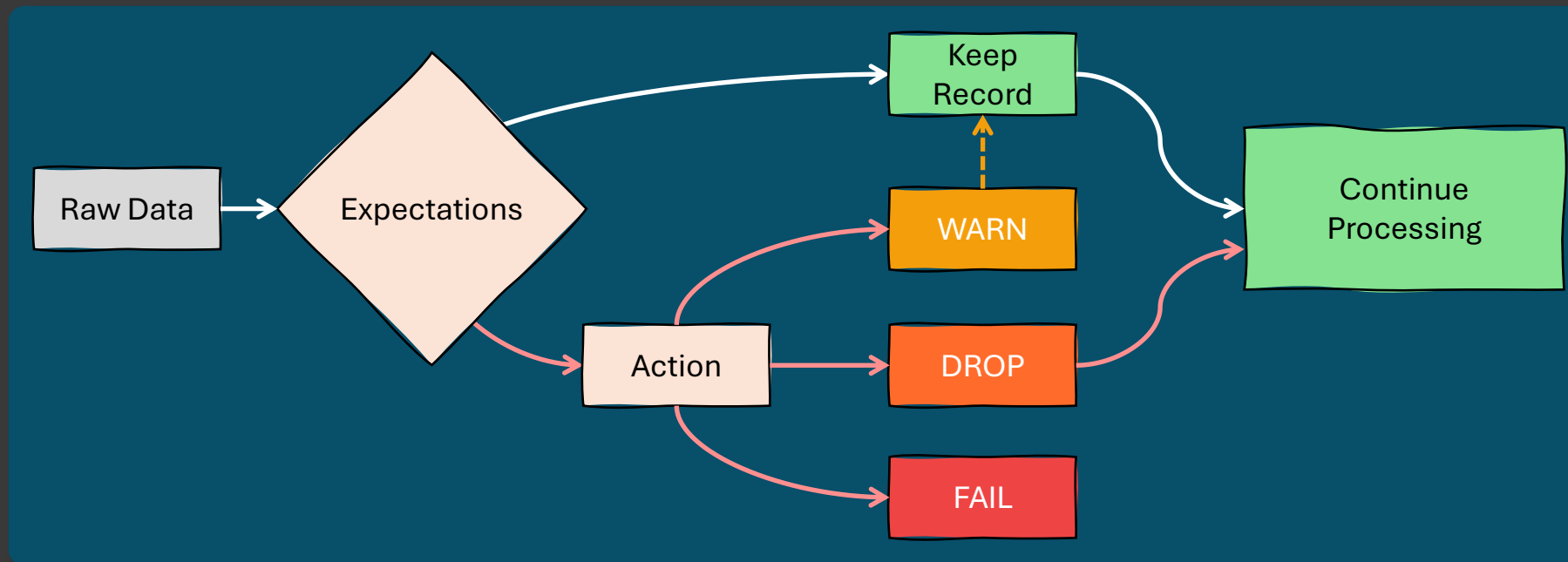
- ❖ Expectations are used to **define** data quality **constraints and rule** on the datasets
- ❖ Guarantees **data arriving** in tables meets those data quality requirements
- ❖ Provides **insights** into data quality for each pipeline update
- ❖ Apply to queries using **Python** or **SQL** constraint clause

```
-- SQL
CONSTRAINT valid_id EXPECT (customer_id IS NOT NULL) ON VIOLATION FAIL UPDATE

-- Python
@dp.expect_or_fail("valid_id", F.col("customer_id").isNotNull())
```



Data Quality Enforcement Flow & Modes

**WARN****EXPECT**

Log the violation. Data still flows.
Use to monitor quality without
stopping ingestion.

DROP**ON VIOLATION
DROP ROW**

Remove the bad record silently.
Metrics track it. Pipeline keeps
running.

FAIL**ON VIOLATION
FAIL UPDATE**

Halt the pipeline update. Nothing
writes until the issue is resolved.



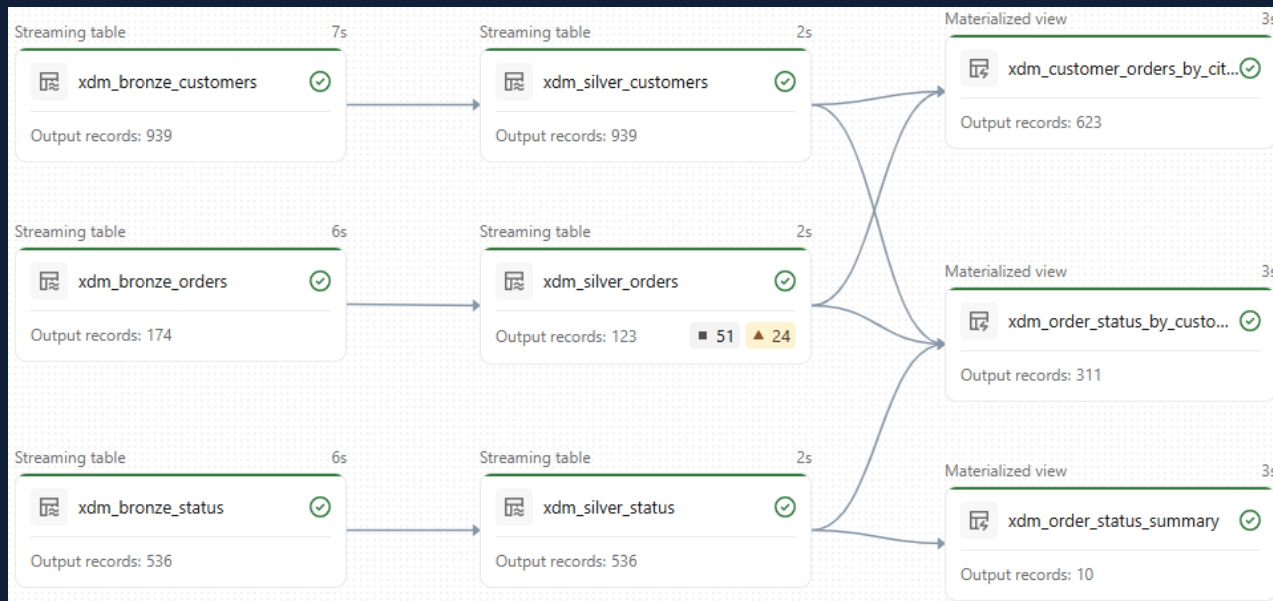
Pipeline metrics show rows warned, dropped, and written — visible in the Databricks UI after each update



Auto Dependency Management

Project Structure

```
src/  
├── 01_bronze/  
│   ├── bronze_customers.sql  
│   └── bronze_orders.sql  
├── 02_silver/  
│   ├── silver_customers.sql  
│   └── silver_orders.sql  
└── 03_gold/  
    └── gold_orders_by_date.sql
```



Dependency Management

- No orchestration code required to connect them
- Each SQL file defines one table or view
- Files reference each other by table name only
- The pipeline scans all files and builds the dependency graph
- Execution order is determined automatically
- New files added are picked up on next deployment



LAKEFLOW CONNECT



Get Your Data In - Without Any Custom Code

 Databricks connectors [Send feedback](#)

 Azure Data Lake Storage Databricks	 Salesforce Databricks	 SQL Server Databricks	 Google Analytics Raw Data Databricks
 SAP Business Data Cloud Databricks	 Workday Reports Databricks	 ServiceNow Databricks	 Zerobus Ingest Databricks Notebook
 Zerobus Ingest REST Databricks Notebook	 SharePoint Databricks Notebook	 Dynamics 365 Databricks Preview	 Foreign Catalog Databricks Preview



What Lakeflow Connect Gives You

- Point-and-click setup — no custom connectors
- Governed by Unity Catalog from day one
- Built-in CDC for database sources
- Deploy and version with DABs (CI/CD ready)



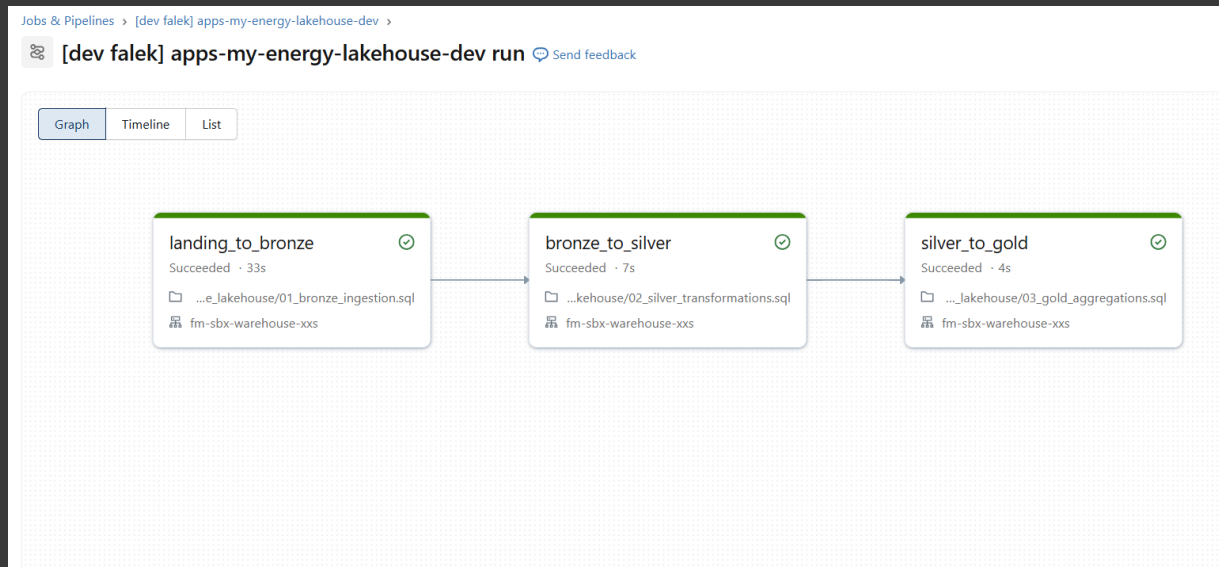


LAKEFLOW JOBS



Lakeflow Jobs

Workflow automation and orchestration for data processing workloads



Serverless Compute

Start fast with fully managed serverless compute. No setup required.



Monitoring & Alerts

Built-in observability: run history, task metrics, email/Slack/webhook notifications.



Complex Control Flow

Conditional branching, loops, retries and dependencies between tasks — all from a visual UI.



Lakeflow Designer

No-code drag-and-drop pipeline builder with built-in natural language interface.

Public Preview



When to reach for Jobs instead Declarative Pipelines

1

Conditional Logic or Loops

SDP builds the DAG from data dependencies. It can't do "if this, then that" or "loop over a list."

Example: if row count is zero, alert and stop. Otherwise, run the transform.

2

Mixing Data with Other Work

SDP is for moving and shaping data in Delta. For notebooks, ML training, dbt, or external tools — use Jobs.

Example: ingest → train ML model → call dbt → send Slack alert.

3

Chaining Multiple Pipelines

One pipeline manages its own DAG. To run pipelines in order, or mix them with notebooks, you need Jobs.

Example: ingestion pipeline → validation notebook → gold pipeline → Tableau refresh.

4

SDP Limits Get in the Way

Source file caps, no pivot(), datasets defined only once, identity columns not supported with AUTO CDC.

Example: a transform that needs pivot() — drop into a notebook task.

👉 **Rule of thumb:** SDP transforms the data. Jobs orchestrates the work.



LAKEFLOW DEPLOYMENT



DECLARATIVE AUTOMATION BUNDLES

Infrastructure as Code for Databricks projects. Formally known as Databricks Asset Bundles.



IaC for Data & AI

Source control, code review, CI/CD - all applied to data projects.



Multi-Target Deploy

Same bundle deploys to dev, staging and production.
One definition, consistent environments.



Team Collaboration

Bundles map to Git repos. Version controlled, repeatable, shareable and can be easily deployed

```
1  # This is a Databricks asset bundle definition for dabs_fm_demo.
2  # See https://docs.databricks.com/dev-tools/bundles/index.html for documentation.
3  bundle:
4    name: dabs_fm_demo
5
6  include:
7    - resources/*.yaml
8
9  targets:
10    dev:
11      # The default target uses 'mode: development' to create a development copy.
12      # - Deployed resources get prefixed with '[dev my_user_name]'
13      # - Any job schedules and triggers are paused by default.
14      # See also https://docs.databricks.com/dev-tools/bundles/deployment-modes.html.
15      mode: development
16      default: true
17      workspace:
18        host: https://adb-0000000000000000.2.azure.databricks.net
19
20    prod:
21      mode: production
22      workspace:
23        host: https://adb-0000000000000000.2.azure.databricks.net
24        root_path: /Workspace/Users/my@email.co.uk/.bundle/${bundle.name}/${bundle.target}
25      permissions:
26        - user_name: my@email.co.uk
27          level: CAN_MANAGE
28      run_as:
29        user_name: my@email.co.uk
30
```



FINAL THOUGHTS & TAKEAWAYS



Try it: Start with one pipeline. One table. See how it feels.



Declare, don't orchestrate

Write what the data should look like. The pipeline handles how to get there — dependencies, retries, incremental logic



Quality is a first-class citizen

EXPECT, DROP ROW, FAIL UPDATE. One clause. Sits inside the table definition, not in a separate job.



Ingestion without the custom code

Lakeflow Connect gives you managed connectors, governed by Unity Catalog from day one. No glue code.



One platform, one experience

Connect, SDP, and Jobs sit together. Ingestion, transformation, orchestration — all in one place.



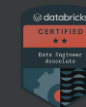
Deploy like software

Databricks Asset Bundles put everything in git. Validate, deploy, promote — same bundle, any environment.

Thank You



Databricks Lakeflow Declarative Pipelines from Code to Flows



Premium sponsors

VENZO_

INSPARI

valantic companies

itm8[®]



Eddytor[™]

data on
Power your data

Standard sponsors

Backstage

Columbus[®]

 **TheVIT**
DATA - ANALYTICS - AI

twoday

Raffle Prizes

